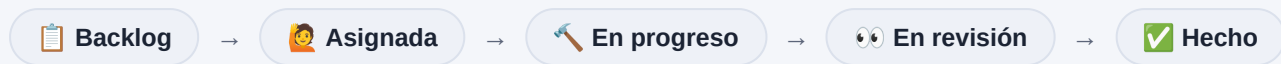


Cómo trabajamos en el repo

Guía paso a paso para todo el equipo, pensada para quien nunca usó ramas ni Pull Requests. Seguíla en orden la primera vez; después es cuestión de repetir los pasos 3 a 7 con cada tarea nueva.



1 Cloná el repo (una sola vez, la primera vez)

Esto se hace UNA vez por persona, no una vez por tarea. Después de esto, el repo ya vive en tu computadora.

```
git clone <url-del-repo>
cd nombre-de-la-carpeta
npm install
```

Si no te corre `npm install`, avisá en el grupo antes de seguir — probablemente falte algo de configuración (Node, o las credenciales de Firebase del README).

2 Elegí una tarea en Trello y asignátela

Andá a la lista **Backlog**, elegí una tarjeta que entiendas y te copa hacer, ponete como miembro de la tarjeta y movela a **Asignada**. Si tenés dudas de qué hace, preguntá en el grupo antes de arrancar — mejor preguntar antes que rehacer.

3 Creá tu rama de trabajo

Una rama es tu propia copia de trabajo: podés romper cosas ahí sin afectar a nadie más. Siempre arranca desde `develop`, nunca desde `main`.

```
git checkout develop
git pull origin develop
git checkout -b tipo/nombre-corto
```

Nombrá la rama así: `feature/pantalla-perfil` para algo nuevo, `fix/boton-transferir` si estás arreglando un bug. Todo en minúsculas, con guiones, sin espacios ni acentos.

Mové la tarjeta de Trello a **En progreso** en este momento.

4 Programá y guardá tu avance

Un *commit* es una foto de tu progreso, con un mensaje que explica qué cambiaste. Hacé commits chicos y seguidos — no dejes todo el trabajo de días en uno solo.

```
git add .  
git commit -m "agrego pantalla de perfil con formulario de alias"
```

Repetí `add + commit` cada vez que termines una parte que funciona. El mensaje va en español, en minúscula, describiendo qué hace el cambio — no "cambios" ni "arreglos".

5 Subí tu rama y abrí un Pull Request

Un *Pull Request* (PR) es pedirle al equipo "revisen esto para sumarlo al proyecto". No se sube nada a `develop` sin pasar por acá.

```
git push origin tipo/nombre-corto
```

Entrá a GitHub: te va a aparecer un cartel amarillo "Compare & pull request" — hacé clic. Verificá que la base sea `develop` (no `main`), escribí un título corto y en la descripción pegá qué hiciste y, si aplica, el link a la tarjeta de Trello.

Mové la tarjeta a **En revisión** y pegá el link del PR en la tarjeta.

6 Un compañero revisa tu código

Alguien del equipo (nunca vos mismo) entra al PR en GitHub, lee los cambios y deja comentarios. Puede **aprobar** directo, o pedir cambios — en ese caso, volvés al paso 4, hacés los ajustes en la misma rama y hacés `git push` de nuevo: el PR se actualiza solo.

Ser revisado no es un examen: es una segunda mirada para agarrar errores antes de que lleguen al proyecto entero. Todos revisamos código de todos en algún momento.

7 Se mergea a develop

Con el PR aprobado, cualquiera con permisos hace clic en "**Squash and merge**" en GitHub (junta todos tus commits en uno solo, más prolijo). `develop` ahora tiene tu cambio.

Mové la tarjeta a **Hecho**. ¡Listo, volvé al paso 2 con la próxima tarea!

8 De develop a main (esto NO lo hace cualquiera)

`main` es la versión "de verdad", la que se entrega. Solo pasa de `develop` a `main` en momentos puntuales (antes de una entrega o demo), lo coordina el equipo, y también por Pull Request — nunca con un push directo. De hecho, el repo va a estar configurado para que sea **imposible** subir algo a `main` sin pasar por ese proceso, así que ni te va a dejar aunque te equivoques.

Glosario en criollo

Rama / branch	Tu copia de trabajo aislada. Cambiás cosas ahí sin romper nada del resto.
Commit	Una foto guardada de tu avance, con un mensaje explicando qué cambiaste.
Push	Subir tus commits desde tu computadora al repo en GitHub.
Pull	Traer a tu computadora los cambios que subieron tus compañeros.
Pull Request (PR)	Pedido formal de "sumen mi rama a <code>develop</code> ", que alguien más tiene que revisar primero.
Merge	Juntar una rama con otra. Es lo que pasa cuando se acepta un PR.
Conflicto	Cuando dos personas cambiaron la misma línea de un archivo de formas distintas y Git no sabe cuál usar. Si te aparece uno y no sabés resolverlo, pedí ayuda antes de forzar nada.

Si algo se traba

No pelees solo contra un error de Git más de 10 minutos. Mandá captura del error al grupo del equipo — casi siempre es algo chico (rama vieja, falta un `git pull`, un archivo que quedó a medio guardar) y se resuelve rápido entre dos.